

Tetris rules and implementations: research for the tetris-js audit

Version 2, October 4, 2026, replaces the earlier 7-page survey. Prepared for David Solis. Target: soliskit/tetris-js at commit 30e0085. Research only. No code changed, no rule adopted, no audit phase run.

Short answer. Your requirements (REQUIREMENTS.md) are the audit's only authority. Published Tetris rules are reference points for finding the places where your game differs, so you can confirm each difference is intentional and written down. Concrete results: (1) your wall kick data matches the published Super Rotation System (SRS) tables in all 16 transitions, and so do the tables in two other public engines I read (NullpoMino and term-tetris); a third (python-tetris) matches in 15 of 16 and has one entry that looks like a sign typo (Section 4). (2) Four public sources agree that a failed rotation fails completely, so your PCE-4 opposite-direction fallback is your own rule, found in none of them. (3) The 2009 Guideline text confirms your lock delay rules, including the "lower row resets the 15 count" detail. I found no published source that makes any of your differences a defect. Conformance to any public version cannot prove the game bug-free or secure, and passing every test cannot either. It only shows the checked claims hold.

Coverage limit. "All publicly available versions" of Tetris is not a finite list. There are hundreds of official releases and clones, and most are closed source. This is a representative sample, not exhaustive. Read in full or in the relevant parts: the 2009 Tetris Design Guideline (a leaked, unauthenticated copy; text from an archive.org OCR, so wording may contain OCR errors); community wiki pages summarizing the Guideline and older games; source code of three public engines (NullpoMino in Java, term-tetris in Rust, python-tetris in Python); one formally proved Tetris (AdaCore SPARK); and web-platform security guidance. Only read at README or summary level: tetr.js, tetr. I did not run any engine, read other engines' tests, or inspect commercial games beyond published descriptions.

1. How to use this in the audit

- **Layer A, your decisions.** What REQUIREMENTS.md says. This decides what counts as a defect. Rule 3 there says behavior it does not describe has no guarantee.
- **Layer B, published references.** The Tetris Guideline (The Tetris Company's spec; the public copy is a leaked 2009 version), historical games, and web-platform security guidance. These are used as independent oracles and as prompts for questions, never as requirements.
- **Differences are not defects.** A difference is classified as: Matches, Documented variation (published games differ among themselves), or Your decision (written in your requirements). Anything you did not write down is "unspecified" and stays unprotected unless you adopt a requirement.

2. What your requirements say (Layer A, from REQUIREMENTS.md at 30e0085)

Grouped by what they govern. IDs are exact.

Area	IDs and the rule you chose
Board and pieces	PCE-1 10 by 20, nothing outside it. PCE-2 seven pieces, standard SRS states, fixed colors. PCE-3 spawn in first rotation, centered, top two rows. PCE-4 SRS kicks in order, then the <i>other</i> direction is tried if all fail. PCE-5 7-bag, fresh bag per game. PCE-6 never out of bounds or overlapping.
Play	PLY-2 gravity $\max(0.25, 0.7 - 0.02 \times (\text{level} - 1))$ s per row. PLY-3 soft drop restarts the gravity timer. PLY-4 hard drop locks at once. PLY-6 lock after 0.5 s, up to 15 resets, count resets on a lower row. PLY-7 hold once per piece. PLY-8 three next pieces.
Scoring	SCO-1 100/300/500/800 for 1 to 4 lines, no level multiplier, no other scoring. SCO-2 level = $\text{floor}(\text{score}/1000) + 1$. SCO-3 high score stored locally.
States and saves	STA-1 to STA-6: new game, pause, game over (a spawn with no room), save on pause and on line clear, pause when page hidden, saved formats that must keep loading.
Safety	SAF-1 saved game is untrusted input, rejected whole. SAF-2 storage failures never stop the game. SAF-3 an engine throw stops the game safely and keeps the last good save. SAF-4 invariants checked after every action and timer. SAF-5 draw and gamepad errors do not stop the loops. SAF-6 long random play holds the rules.
Input, display, app	INP-1 to INP-6 (keys, hold-to-repeat 167 ms then 33 ms, modifier keys, gamepad, touch, hold box as a button). DSP-1 to DSP-8. APP-1 to APP-6, including APP-6: GitHub Pages shares one origin across a user's projects, so the game touches only its own storage and caches.
Process	QA-1 to QA-6: every requirement tested, 100% unit coverage of <code>public/game/</code> , 100% browser coverage of <code>public/script.js</code> , WebKit at iPhone size, strict typecheck, no surviving mutants in the engine.

3. Published references, compared with your rules (Layer B)

Column "Class": **Matches**, **Variation** (published games differ among themselves), **Yours** (your requirement states a different rule; intent is documented, so not a defect), **Unspecified** (neither your requirements nor the test suite describe it).

Topic	Published (source)	tetris-js	Class
Playfield	2009 Guideline sections 3.4 and glossary: 10 x 20 matrix plus a 20-row invisible buffer zone; pieces generated on rows 21 and 22. SRS page: spawn row varies by game, from rows 22-23 in Tetris Worlds to rows 20-21 in Tetris Effect.	PCE-1: exactly 10 x 20, positions outside hold nothing. PCE-3 spawn in the top two rows.	Variation, and Yours. Consequence: no hidden buffer, so kicks above the top are blocked and "lock out" cannot occur (see top-out row).

Rotation states, spawn orientation	2009 Guideline 3.4: three-wide pieces generated on cells 4 to 6 of a 10-wide matrix, north facing (SRS page: I and O centered).	PCE-2. Spawn column from spawned (): 3-wide pieces at columns 3 to 5, I at 3 to 6, O at 4 to 5 (zero-based).	Matches by my reading of the code: columns 3 to 5 zero-based are cells 4 to 6.
Wall kicks	Five tests per transition; JLSTZ share one table, I has its own, O does not kick (SRS page). 2009 Guideline Appendix A: "if all five rotations fail, the rotation cannot occur in that situation." NullpoMino, term-tetris and python-tetris also leave the piece unchanged on failure (Section 4).	Same tables, verified in Section 4. After all five fail in the asked direction, PCE-4 tries the opposite direction.	Tables match. Opposite-direction fallback is Yours: contradicted by the Guideline text and absent from all three engines I read.
Reversibility	Guideline summary: all rotations are reversible, one possible means the opposite is also possible (Guideline page).	Counterclockwise uses the negated clockwise kicks of the target state (code comment and rotate ()).	Consistent with how SRS tables are built. Section 4 checks the numbers.
Randomizer	Random Generator: shuffle one of each of 7, deal, refill (Random Generator page). Some games use other bags or the TGM randomizer.	PCE-5 7-bag; Fisher-Yates shuffle in tetrominoFactory.js; first seven of a game all different.	Matches.
Lock delay	2009 Guideline 5.7: Extended Placement is the default; 0.5 s timer; reset on move or rotate; 15 left/right moves or rotations; the count resets if the piece falls one row below the lowest row yet reached; after 15, it locks at once on touching a surface.	PLY-6: 0.5 s, 15 resets, count resets when a lower row is reached.	Matches, including the lower-row reset and immediate lock after 15. One nuance differs: the 2009 text says a rotation that lifts the piece off its surface pauses the timer without resetting it, and only a lower landing resets it. Your PLY-6 cancels and restarts the delay. Not a defect (rules 1 and 3); worth knowing if a reviewer compares.
Gravity	Guideline marathon curve: seconds per row = $(0.8 - (\text{level} - 1) \times 0.007)^{(\text{level} - 1)}$; level 1 is 1/60 G, about 1.0 s per row (Marathon page).	PLY-2: 0.7 s at level 1, 0.02 s faster per level, floor 0.25 s (reached at level 24, 23,000 points).	Yours. Not the Guideline curve; never reaches 20G or any "kill" speed.
Leveling	Guideline: level rises only by clearing lines, 10 per level in fixed-goal marathon; variable goal in earlier games (Marathon page, Guideline page).	SCO-2 level from score / 1000.	Yours.

Scoring	Guideline-era: 100/300/500/800 x level, plus T-spins, back-to-back x 1.5, combos, soft drop 1 per cell, hard drop 2 per cell, perfect clear bonuses (Scoring page, an aggregator-style wiki, "accurate for games that use 3-corner T-spin"). NES: 40/100/300/1200 x (level + 1) (NES Wikipedia page).	SCO-1: 100/300/500/800, no multiplier, no drop points, no T-spin, combo or back-to-back.	Yours. Base values match the Guideline's; everything else is absent by choice. Note SAF-4 requires score divisible by 100, so any drop-point scoring would need to change that invariant first.
Hold	2009 Guideline 5.6: one held piece; the swapped-in piece starts from its generation position, north facing; a lock down must take place between holds.	PLY-7 hold once per piece, piece returns in its starting state.	Matches.
Next queue, ghost	Guideline: next queue shown, 3 to 6 in published games; ghost piece required, often togglable (Guideline page, differences page).	PLY-8 three; PLY-5 ghost, no toggle stated.	Matches within the published range.
Game over	2009 Guideline section 10: game over on lock out (whole piece locks above the skyline), block out (a starting cell of the next piece is blocked), or top out (line attacks). Other games use other variants (Top out page).	STA-3: game over only when a new or held piece has no room (block out).	Yours. Block out matches. With no hidden zone, lock out cannot arise; top out is a multiplayer rule.
Controls	Guideline keyboard: arrows shift, Up rotates clockwise, Down soft drop, Space hard drop, C or Shift hold, Z or left Ctrl counterclockwise (Guideline page).	INP-1: A/D or arrows, W/Up clockwise, Z counterclockwise, S/Down soft, Space hard, H hold.	Yours (hold key differs: H, not C or Shift).
Auto-repeat	Guideline recommendation: DAS 167 ms, ARR 33 ms (Guideline page, "recommended but non-mandatory").	INP-2: 167 ms then 33 ms.	Matches the recommended values.
Soft drop	2009 Guideline 5.5: soft drop is 20 times faster than the normal fall speed; does not lock until the lock timer ends.	PLY-3 one row per press, 50 ms repeat (INP-2).	Variation. At your level 1 gravity (0.7 s) 20x would be 35 ms; yours is 50 ms.

Entry delay, line-clear delay, T-spin, garbage, perfect clear, DAS cut	Present in many Guideline games.	Not in requirements.	Unspecified. Absence is not a defect (rule 3).
---	-------------------------------------	----------------------	---

Historical systems, for context

- **NES Tetris (1989):** Nintendo Rotation System, no wall kicks, no lock delay. Level 29 doubles gravity to one row per frame on NTSC. The game can crash after about 1,550 lines from inefficient multiplication; first reached December 21, 2023 (NES Tetris Wikipedia page).
- **Other rotation systems:** Nintendo, Sega, Atari, Arika, SRS (StrategyWiki, TetrisWiki). They differ in kicks, floor behavior and pivot points. Arika's SRS variant changes I-piece kick data so it is symmetric (SRS page).
- **Guideline versions:** a yearly spec, known publicly from one 2011 leak of the 2009 version; later changes are inferred from games (Guideline page). The 2009 text is a leaked copy, so "the Guideline" as a requirement can never be fully verified.

4. Public implementations read, and the kick-table comparison

Method: I fetched the source files named below from their public repositories, converted each kick table to the wiki's (x, y) convention (y positive up), and compared all 8 transitions per table with the SRS tables at tetris.wiki/SRS. I did this by script for each implementation and for yours (tetrominoFactory.js and tetromino.js at 30e0085, with counterclockwise kicks derived as rotate() does). This is a check of the data, and of how each engine handles a failed rotation. I did not run any of the engines.

Implementation	What I read	Kick data vs wiki SRS	If every kick fails	Other points
tetris-js (yours)	tetrominoFactory.js, tetromino.js	16 of 16 match (JLSTZ 8, I 8; O has one entry)	Tries the opposite direction (PCE-4)	Your rule; documented and tested.
NullpoMino (Java, open source, 2008 to 2011)	StandardWallkick.java, BaseStandardWallkick.java in nullpomino/nullpomino	16 of 16 match (its y axis points down; converted)	Returns no result, so rotation does not happen	Also has 180-degree kick tables (not in the 2009 Guideline text), an allowUpward flag that permits upward kicks only when set, and other wallkick classes for Classic, GBC and others. Its rule presets (STANDARD, CLASSIC) show how one engine separates rule sets.
term-tetris (Rust terminal game, hobby project)	src/tetromino.rs	16 of 16 match; its table heading reads like the wiki's	Restores the old rotation and returns None	Small project. Agreement with the wiki here shows copying fidelity, not independent derivation.

python-tetris (Python library, README says "guideline-compliant by default")	tetris/impl/rotation.py in g3ner1c/python-tetris	15 of 16 match. I-piece kicks for 2 to R: the fifth test is (row -1, col +2) in the file; the wiki converts to (row -1, col -2). Its own L to 0 entry, which the wiki table makes identical, has (-1, -2).	Piece unchanged	Looks like a sign typo in one table entry. I did not run it, read its tests, or report it upstream. Treat it as a lead, not a confirmed bug.
AdaCore SPARK Tetris (Ada, formally analyzed)	Two AdaCore blog posts; the code archive was not opened	Not checked	Not checked	Built from Wikipedia's rules and a web page on SRS. The article's proof work is absence of run-time errors (array bounds, ranges) with loop invariants and flow analysis on about 165 lines of game logic. Rule correctness is a different property and the article does not claim it.
tetr.js (JavaScript), tetr (Rust)	README only	Not checked	Not checked	tetr.js README states Guideline compliance, SRS. tetr says it was inspired by NullpoMino. Claims, not verified.
generated-tetris specification.md	Whole file	n/a	n/a	A repository spec that says it follows the official Guidelines. I did not use it as a source: its claim is self-asserted and unverified.

What this shows. (a) Your tables agree with the published tables. (b) Agreement among implementations is weak evidence: they copy the same wiki or Guideline, so three matches are not three independent checks. (c) A published-table comparison does catch transcription slips, as in python-tetris, and hand-checked suites with 100% coverage can miss them because coverage measures that a line ran, not that a number is right. That is why the audit's circular-oracle concern applies to kick data. (d) **Failure behavior:** the Guideline and all three engines I read treat a fully blocked rotation as no rotation. None tries the opposite direction.

Not shown. Whether your rotation logic applies the data correctly, collision at walls and the floor, and how the PCE-4 fallback behaves in play. Data tables are one input to those proofs.

5. Differences worth your decision, not defects

All are already explicit in REQUIREMENTS.md except where noted. They are listed because a reviewer comparing you to the Guideline will see them first. None needs action for the audit unless you want it.

1. **PCE-4 opposite-direction fallback.** Pressing rotate can turn the piece the other way. The 2009 Guideline text and all three engines I read say a failed rotation fails completely. The behavior is deliberate and tested. It is the most unusual rule against published sources.
2. **No hidden buffer rows (PCE-1).** Standard SRS kicks that need to go above row 1 are impossible, which changes some rotations at the top of the stack compared with Guideline games.
3. **Level from score (SCO-2) and your own gravity curve (PLY-2).** Players used to marathon mode will see different pacing.
4. **Scoring (SCO-1).** No level multiplier. Fine, and simpler to prove.
5. **Lock delay when a rotation lifts the piece (PLY-6).** The 2009 text pauses the timer and does not reset it; yours restarts the delay. The lower-row reset and immediate lock after 15 are confirmed.

6. Security and robustness references that bear on your requirements

Source	What it says	Bears on
OWASP HTML5 Security Cheat Sheet, Local Storage	Data in <code>localStorage</code> is shared across the whole origin, can be loaded with malicious data by a single XSS, and should not be considered trusted. No path scoping exists, and the sheet advises against hosting multiple applications on one origin.	SAF-1 (saved game treated as untrusted), SAF-2, APP-6 (shared GitHub Pages origin, own key names). Your design already follows this guidance. It is one more reason the audit keeps the "who else can write this origin's storage" question open.
MDN, <code>Window.localStorage</code>	Values are strings; access can throw if blocked by policy; data persists with no expiration; origin-wide.	SAF-2 (missing, blocked, full, throwing).
Chromium Service Worker Security FAQ	A service worker registration lasts indefinitely, the update check happens on navigation, and sites should be careful about which paths can serve worker scripts. Registration matching is by serialized URL prefix.	APP-2, APP-3, APP-6: <code>sw.js</code> at <code>/tetris-js/</code> on a shared origin is a trust boundary worth stating in Phase 1.
Vittorio Romeo, "implementing secure leaderboards" (Open Hexagon, 2021; not Tetris)	A client-sent score can be spoofed; the author makes gameplay deterministic from a seed plus an input log, and the server replays it. A compiler flag (<code>-ffast-math</code>) once caused replay desyncs.	Your game has no server or leaderboard, so score spoofing is out of scope (F17 already records that a hand-edited stored high score is accepted). The relevant idea is determinism: your random source and scheduler are injectable (PCE-5), which makes seeded replay a possible audit instrument.

I did not find a published security standard for Tetris games specifically. For a client-only game with no accounts, the realistic classes are: untrusted stored data, storage faults, a malicious or stale service-worker cache, same-origin neighbors on GitHub Pages, and input and timing faults. Your SAF and APP requirements already name these, so the audit question is evidence, not coverage of new topics.

7. Bug history and verification approaches

- **Numeric limits:** NES Tetris stops rendering scores above 999,999, and its multiplication code can crash it after about 1,550 lines. Relevance: your score is validated with `Number.isSafeInteger` and a multiple of 100, and level is unbounded, so overflow is not reachable in play. That is a reasoned claim to confirm in Phase 1, not a finding.
- **Kick table errors** occur in practice: `python-tetris` has one I-piece entry that disagrees with the published table (Section 4, a lead I did not run). `TetrisWiki` also documents deliberate I-piece differences in Arika's SRS variant. Your tables match the Guideline version.
- **Formal work exists, but I found only one Tetris example:** AdaCore's SPARK Tetris, which proves absence of run-time errors (not rule correctness) in about 165 lines of Ada. It shows proof is feasible for a small core and says nothing about your JavaScript.

- **Differential oracles:** NullpoMino's wallkick classes and the 2009 Guideline text are independent enough of your code to serve as references for kicks, hold, lock down and game-over rules. I read NullpoMino's wallkick source only, not its lock delay or hold code, and ran nothing.

8. Suggested inputs for Phase 1 (research outputs, not decisions)

- Record the published SRS tables and the Section 4 comparison as the oracle for your kick data. Add the 16-transition check to the evidence for PCE-2 and PCE-4 only if you adopt that proof obligation.
- Decide whether to record, in AUDIT.md, the intentional differences in Section 5 as "not a finding". That follows your own rule that rejected findings stay in the ledger.
- Use published rules only as hypotheses. A difference becomes a finding only when it breaks a requirement or you adopt a new one in the same commit.
- If you want an independent oracle beyond your own tests, the options are a published-table check, a second implementation from the SRS spec, or differential testing against an open-source engine. Each proves a different thing. Choosing is yours.

9. Sources

Your repository: soliskit/tetris-js at 30e0085, files REQUIREMENTS.md, AUDIT.md, public/game/tetrominoFactory.js, tetromino.js, gameManager.js (read through the GitHub connection). github.com/soliskit/tetris-js

- Tetris Guideline: tetr.wiki/Tetris_Guideline
- Super Rotation System and kick tables: tetr.wiki/SRS
- Random Generator: tetr.wiki/Random_Generator
- Lock delay: tetr.wiki/Lock_delay
- Scoring: tetr.wiki/Scoring
- Marathon and speed curve: tetr.wiki/Marathon
- Top out: tetr.wiki/Top_out
- 2009 Tetris Design Guideline (leaked, unauthenticated copy; I read the OCR text of "2009 Tetris Design Guideline_chocr.html.gz"): archive.org/details/2009-tetris-variant-concepts_202201. Sections cited: 2.5.4, 3.3, 3.4, 5.3 to 5.7, 10, Appendix A1.4, glossary.
- NullpoMino wallkick source: [StandardWallkick.java](#) and BaseStandardWallkick.java in the same folder
- term-tetris: [src/tetromino.rs](https://src.tetromino.rs)
- python-tetris: github.com/g3ner1c/python-tetris, file tetris/impl/rotation.py (class SRS, table i_kicks)
- AdaCore: [Tetris in SPARK on ARM Cortex M4](#)
- Vittorio Romeo, secure leaderboards: vittorioromeo.com/index/blog/oh_secure_leaderboards.html
- tetr.js README: github.com/simonlc/tetr.js; tetr README: github.com/tiehuis/tetr (README level only)
- generated-tetris spec (read, not relied on): github.com/VaclavSynacek/generated-tetris

- Guideline compliant game differences: tetris.wiki/Guideline_compliant_game_differences
- Rotation systems: strategywiki.org/wiki/Tetris/Rotation_systems
- NES Tetris: [en.wikipedia.org/wiki/Tetris_\(NES_video_game\)](https://en.wikipedia.org/wiki/Tetris_(NES_video_game)); [tetris.wiki/Tetris_\(NES\)](https://tetris.wiki/Tetris_(NES))
- NullpoMino: tetris.wiki/Nullpomino (summary page only)
- OWASP HTML5 Security Cheat Sheet: cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html
- MDN localStorage: developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
- Chromium Service Worker Security FAQ: chromium.googlesource.com/.../service-worker-security-faq.md
- AdaCore, SPARK Tetris on the Raspberry Pi Pico: adacore.com/blog/spark-tetris-on-the-raspberry-pico

Not covered: licensing and trademark (the Guideline ties compliance to the Tetris logo and music; irrelevant to a portfolio game unless branded as Tetris), commercial games' behavior beyond published descriptions, running any other engine, other engines' lock delay, hold and scoring code or tests, the Guideline's kick-table figures themselves (the OCR text of the diagrams is not usable; kicks were compared against the wiki), and device or Safari-specific behavior. The 2009 Guideline is a leaked copy of a confidential document and may differ from later versions; the Tetris Company's current rules are not public.